Modern Architectures for Language Models

Assignment #1

Long Context and Mixture of Experts

Assignment overview and motivation

- *Format: Individual or group (max 3 students) with offline defense*
- *Grading: 20 points (+5 bonus points for the optional Part 3)*
- *Timeline: Assignment #1 available in Week 4, Code submission deadline till Week 6, offline defense deadline till Week 7*

A pretrained checkpoint comes with two properties that practitioners routinely change after pretraining: the range of positions its rotary embeddings were trained on, and the way its router spreads tokens over experts. Both look like one line in a config file, and both fail quietly. A model pushed past its training length can keep a reasonable average perplexity while its predictions at far positions fall apart, and a mixture-of-experts model can keep a good loss while most of its tokens go to a handful of experts.

In this assignment you will take small open checkpoints and change both properties yourself. In Part 1 you extend the context window of a pretrained model with rotary scaling methods, with and without fine-tuning, and measure where quality breaks relative to the training length. In Part 2 you instrument the router of a mixture-of-experts model on Ukrainian text, then continue its training under different load-balancing strategies and measure what each strategy does to balance and to language-modeling quality. Part 3 (optional) moves the same balancing question inside a parameter-efficient fine-tuning method.

The Ukrainian corpora and the SFT tasks are the ones you already used in the NLP course (Assignment #1: Tokenization, Embeddings; Assignment #2: Seq2Seq; Assignment #3: LLMs: SFT + Alignment), so you can reuse your data pipelines.

This assignment focuses on:

- *understanding why rotary position embeddings fail beyond the training length, and what position interpolation, NTK-aware scaling and YaRN change in the frequency spectrum*
- *measuring long-context quality honestly: position-binned loss and retrieval, not a single averaged perplexity*

- *understanding routing collapse and the three standard remedies: the auxiliary load-balancing loss, the router z-loss and the auxiliary-loss-free bias update*
- *running controlled comparisons under a realistic single-GPU budget, with matched baselines and pre-registered predictions*

Learning objectives

By completing this assignment, students will be able to:

- *Implement position interpolation, NTK-aware scaling, dynamic NTK and YaRN by editing the rotary frequency table directly, and verify them against a reference implementation*
- *Evaluate a context-extended model with loss binned by position and with a passkey retrieval test, in Ukrainian and in English*
- *Fine-tune a pretrained model at a longer context under T4 memory limits and compare it with training-free scaling*
- *Compute the KV-cache memory of a model from its config and reconcile the prediction with the measured allocation*
- *Instrument a mixture-of-experts router: expert load, MaxVio, load entropy and routing drift, per layer and per language*
- *Implement and compare the auxiliary load-balancing loss, the router z-loss and the auxiliary-loss-free bias update under continued pretraining*
- *(Bonus) Implement a mixture of LoRA experts and compare it with plain LoRA at matched trainable parameters*

Assignment deliverables:

- *A reproducible repo (or notebook + final module) with:*
  - *data preprocessing and train / validation / test splits*
  - *rotary scaling module (your own implementation) and its equivalence tests*
  - *evaluation scripts (position-binned loss, bits per byte, passkey retrieval)*
  - *fine-tuning scripts for Part 1 and continued-pretraining scripts for Part 2*
  - *router instrumentation and the balancing implementations*
  - *clearly saved configs and random seeds*
- *A short report (4-8 pages or jupyter / README equivalent) including:*
  - *model choice and parameter count (total and active for mixture-of-experts models)*
  - *hardware / compute used and training time for every run*
  - *all important hyperparameters*
  - *your pre-registered predictions and the prediction-error table*
  - *tables / plots with metrics*
  - *failure cases and negative results*
- *A config file (config.yaml or equivalent) with default reproducible settings*
  - *A lockfile (poetry.lock or uv.lock) and package metadata (pyproject.toml)*
  - *(Optional) Dockerfile or container instructions*
- *Every claim in the report must be backed by a measurement. Claims without measurements score zero for the corresponding part.*

Task

Datasets:

Ukrainian text (the corpora from NLP Assignment #1):

- *UberText 2.0: https://lang.org.ua/en/ubertext/ (news, fiction, social, wikipedia and court subcorpora)*
- *Malyuk: https://huggingface.co/datasets/lang-uk/malyuk*
- *Kobza: https://huggingface.co/datasets/Goader/kobza*

English long documents:

- *PG-19 test split: https://huggingface.co/datasets/emozilla/pg19-test (be aware that Project Gutenberg books are present in the pretraining data of most open models)*

Parallel sentences for Ukrainian vs English comparisons (the translation dataset from NLP Assignments #2 and #3):

- *FLORES-200: https://huggingface.co/datasets/SEACrowd/flores200*

SFT tasks for the optional Part 3 (the tasks from NLP Assignments #2 and #3):

- *Spivavtor: https://huggingface.co/datasets/grammarly/spivavtor*
- *UberText-GEC: https://huggingface.co/datasets/lang-uk/UberText-GEC*
- *XL-Sum: https://huggingface.co/datasets/csebuetnlp/xlsum*
- *FLORES-200 (above)*

If you face computational limits, use a reasonable subset of each corpus. Document the subset and keep all evaluation documents disjoint from all training documents.

Model selection

You may use any open model of your choice, provided that it satisfies the requirements of the corresponding part and that you justify the choice in the report: native context length, rotary configuration, number of experts and top-k, language coverage (in particular tokenizer fertility on Ukrainian), parameter count and compute feasibility. The models below are verified starting points, not a restriction.

***Part 1** requires a model with a short native context (at most 4,096 tokens), so that a 4x extension still fits on a T4:*

- *EuroMoE-2.6B-A0.6B: https://huggingface.co/utter-project/EuroMoE-2.6B-A0.6B-2512 (mixture of experts, 4,096 tokens, rotary base 500,000, Ukrainian in the pretraining data; can be used for both Part 1 and Part 2)*
- *TinyLlama-1.1B: https://huggingface.co/TinyLlama/TinyLlama-1.1B-intermediate-step-1431k-3T (dense, 2,048 tokens, rotary base 10,000)*
- *OLMo-1B-0724: https://huggingface.co/allenai/OLMo-1B-0724-hf (dense, 4,096 tokens, rotary base 10,000, English only)*
- *Granite-3.0-1B-A400M: https://huggingface.co/ibm-granite/granite-3.0-1b-a400m-base (mixture of experts, 4,096 tokens)*

Models with a long native context (e.g. Qwen2.5-0.5B with 32,768 tokens, Llama-3.2-1B with 131,072 tokens) are useful as a contrast, but not as the main Part 1 model: their extended range does not fit into T4 memory.

***Part 2** requires a mixture-of-experts model whose router you can access and train:*

- *EuroMoE-2.6B-A0.6B: https://huggingface.co/utter-project/EuroMoE-2.6B-A0.6B-2512 (2.6B total / 0.6B active, 64 experts, top-8, softmax router; T4)*
- *Granite-3.1-1B-A400M: https://huggingface.co/ibm-granite/granite-3.1-1b-a400m-base (1.3B total / 0.4B active, 32 experts, top-8; Ukrainian is not among its training languages, which makes it a useful contrast; T4)*
- *Qwen1.5-MoE-A2.7B: https://huggingface.co/Qwen/Qwen1.5-MoE-A2.7B (14.3B total / 2.7B active, 60 routed experts, top-4, plus a shared expert with its own sigmoid gate; fits on 2 x T4 in float16 for evaluation, or with 4-bit experts for training the router and attention)*
- *Qwen3-30B-A3B-Base: https://huggingface.co/Qwen/Qwen3-30B-A3B-Base (30.5B total / 3.3B active, 128 experts, top-8, Ukrainian among the supported languages; realistic on an A100-40GB with 4-bit experts)*
- *OLMoE-1B-7B: https://huggingface.co/allenai/OLMoE-1B-7B-0125 (6.9B total / 1.3B active, 64 experts, top-8, fully open training data and logs; A100)*

Notes on 4-bit loading of mixture-of-experts models: in recent transformers releases the experts of several model classes are stored as fused 3-D tensors that bitsandbytes does not quantize, so check the actual memory footprint after loading. Keep the router, the shared-expert gate and the attention layers in 16-bit (e.g. with `llm_int8_skip_modules`) if you train them.

***Part 3** uses the model from your NLP Assignment #3, or any instruction model up to about 3B parameters (e.g. Qwen2.5-1.5B-Instruct).*

Part 1 - Extending the context window of a RoPE model (Lecture 2)

Let L_train be the native context length of your model and s the scale factor. The target is s = 4 (e.g. 4,096 → 16,384 tokens, or 2,048 → 8,192 tokens).

Task:

1. *Evaluation set. Build a held-out set of at least 100 Ukrainian documents (e.g. court decisions, fiction, long Wikipedia articles from UberText 2.0 or Kobza) each at least s·L_train tokens long under your tokenizer.*
2. *Metrics. Implement:*
    a. *teacher-forced next-token loss as a function of position, averaged in bins (e.g. bins of L_train/4 up to s·L_train); plot it with a vertical line at L_train*
    b. *passkey retrieval (RULER-style): a random 5-digit key hidden at depths of 10%, 50% and 90% of a filler document, queried at the end, at lengths L_train, 2·L_train and 4·L_train*
3. *Training-free scaling. Implement the following methods yourself by modifying the inverse-frequency table (or the cos / sin cache), and verify each against the transformers `rope_scaling` implementation where one exists (maximum absolute difference of the frequency table below 1e-5):*
    a. *no scaling (plain extrapolation, the baseline)*
    b. *position interpolation (linear scaling)*
    c. *NTK-aware base scaling (there is no static NTK type in transformers, so derive and implement b' yourself)*

Success criteria:

- *A loss-vs-position plot with all methods and a vertical line at L_train.*
- *A short write-up answering: which method would you ship and do perplexity and passkey retrieval agree?*

NOTES:

- *Do not compute perplexity on text generated by the model itself. The model assigns high probability to its own samples, and repetitive degenerate output scores deceptively well. Always use teacher-forced loss on held-out human-written text.*
- *Dynamic NTK recomputes the scale from the current sequence length, so a single 16k forward pass evaluates every position with the 16k scale. Evaluate it on separate prefixes (L_train, 2·L_train, 4·L_train).*
- *Force the memory-efficient attention kernel with* `torch.nn.attention.sdpa_kernel`*. A silent fallback to the math kernel materializes the full attention matrix and runs out of memory at 16k.*
- *Print the frequency table after loading the model to confirm that your scaling was applied. Configuration keys for rotary scaling changed between transformers versions (*`rope_scaling` *vs* `rope_parameters`*), so pin your version.*

Core papers:

- *RoFormer: Enhanced Transformer with Rotary Position Embedding (Su et al., 2021): https://arxiv.org/abs/2104.09864*
- *Extending Context Window of Large Language Models via Positional Interpolation (Chen et al., 2023): https://arxiv.org/abs/2306.15595*
- *YaRN: Efficient Context Window Extension of Large Language Models (Peng et al., 2023), which also formalizes NTK-aware and dynamic NTK scaling: https://arxiv.org/abs/2309.00071*
- *LongLoRA: Efficient Fine-tuning of Long-Context Large Language Models (Chen et al., 2023), for trainable embeddings and normalization layers: https://arxiv.org/abs/2309.12307*
- *Landmark Attention (Mohtashami and Jaggi, 2023), the origin of the passkey retrieval test: https://arxiv.org/abs/2305.16300*
- *RULER: What's the Real Context Size of Your Long-Context Language Models? (Hsieh et al., 2024): https://arxiv.org/abs/2404.06654*
- *Can Perplexity Reflect Large Language Model's Ability in Long Text Understanding? (Hu et al., 2024): https://arxiv.org/abs/2405.06105*

Part 2 - Load balancing in mixture-of-experts models (Lecture 3)

Task:

1. *Model card. Report the number of routed and shared experts, top-k, the gating function (softmax or sigmoid, normalization of the selected weights), the balancing method and coefficients used in pretraining (if published), and total vs active parameters.*
2. *Data. Take a Ukrainian corpus from AI 340 NLP Assignment #1 and create train / validation / test splits.*
3. *Instrumentation. Collect router logits for every layer (e.g. with* `output_router_logits=True` *or forward hooks) and compute, per layer:*
    a. *expert load fractions f_i (the share of routed token slots assigned to expert i)*

   b. *MaxVio = (max_i load_i − mean load) / mean load, both globally on a held-out set and per training batch*
   c. *normalized entropy of the load distribution*
   d. *the router log-partition logsumexp(ℓ) (the quantity penalized by the z-loss)*
4. *Noise floor. Compute the MaxVio of a uniformly random top-k router on the same number of tokens. Differences below this floor are not evidence of anything.*
5. *Base model analysis. Identify overloaded and rarely used experts, and compare early, middle and late layers.*
6. *Continued pretraining. Continue next-token-prediction training on the Ukrainian train split with the same data order, token budget, seed and hyperparameters for every strategy:*
   a. *no balancing (auxiliary coefficient 0), the control*
   b. *auxiliary load-balancing loss (Switch / GShard form) with at least two coefficients, e.g. 1e-3 and 1e-2*
   c. *auxiliary-loss-free balancing: a per-expert bias added to the router scores for top-k selection only, updated with a sign rule after every optimizer step (implement it yourself)*
   d. *(optional) router z-loss added to one of the strategies above*
   e. *a frozen-router control (router learning rate 0)*
7. *After training, report for every strategy: perplexity, per-layer global MaxVio and load entropy, routing drift from the base model (e.g. the mean Jaccard overlap between the base and the trained top-k sets of the same tokens), and the curves of the balance metrics during training.*

Success criteria:

- *A table: strategy × {test perplexity, mean and max global MaxVio over layers, load entropy, routing drift}, with the random-router noise floor.*
- *Layer × expert load heatmaps for the base model and every strategy.*
- *A short write-up answering: which strategy balances best, what does it cost in language-modeling quality.*
- *Small or null differences in loss between strategies are an expected outcome of a short continued-pretraining run. Report them with at least 2 seeds for the key comparison instead of over-interpreting them.*

NOTES:

- *Before training, check that the auxiliary loss is actually computed: assert that it is non-zero and that it changes when you change the coefficient. In some model classes and library versions the router logits are not returned by default, and the loss silently disappears. In transformers, the coefficient must be passed to* `from_pretrained` *(*`router_aux_loss_coef=...`*); changing the config after loading has no effect.*
- *The built-in* `load_balancing_loss_func` *in transformers equals k (not 1) at perfect balance for top-k routing. Check the normalization before comparing coefficients with the papers.*
- *Store the balancing bias as a float32 buffer outside the optimizer. It must change only which experts are selected, never the gate values used to mix the expert outputs. With two GPUs, sum the expert counts across devices before the bias update.*
- *With gradient checkpointing, use* `use_reentrant=False`*. Otherwise tensors saved for a loss outside the checkpointed forward may not receive gradients.*
- *Compare models with different tokenizers in bits per byte, not in token perplexity.*

Core papers:

- *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer (Shazeer et al., 2017): https://arxiv.org/abs/1701.06538*
- *GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding (Lepikhin et al., 2020): https://arxiv.org/abs/2006.16668*
- *Switch Transformers (Fedus et al., 2021), the auxiliary loss in the form $N \cdot \Sigma\ f_i \cdot P_i$: https://arxiv.org/abs/2101.03961*
- *ST-MoE: Designing Stable and Transferable Sparse Expert Models (Zoph et al., 2022), the router z-loss: https://arxiv.org/abs/2202.08906*
- *Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts (Wang et al., 2024): https://arxiv.org/abs/2408.15664*
- *DeepSeek-V3 Technical Report (DeepSeek-AI, 2024), sigmoid gating with bias-based balancing: https://arxiv.org/abs/2412.19437*
- *OLMoE: Open Mixture-of-Experts Language Models (Muennighoff et al., 2024): https://arxiv.org/abs/2409.02060*

Part 3 (Optional, +5 bonus points) - Mixture of LoRA experts

Disclaimer. *The idea looks promising, but the existing framework is legacy. The reference implementation of MixLoRA and the MoE-PEFT framework it depends on pin old versions of torch, transformers and peft that conflict with current Kaggle and Colab images, and the Hugging Face PEFT library has no built-in MixLoRA-style tuner. Expect to implement the module yourself or to write a significant amount of custom code. A clean from-scratch implementation is roughly 150-250 lines.*

Task:

1. *Take your SFT pipeline from NLP Assignment #3 (or an instruction model up to about 3B parameters) and one Ukrainian task from NLP Assignments #2 and #3 (e.g. grammatical error correction on Spivavtor or UberText-GEC).*
2. *Implement a MixLoRA-style layer: E LoRA experts inside every feed-forward block (on the gate, up and down projections) that share the frozen feed-forward weights, a top-k router, and plain LoRA on the attention projections.*
3. *Balancing: train with no balancing, with the auxiliary loss and with the bias update from Part 2 (reuse your code).*
4. *Baselines: plain LoRA at rank 16, plain LoRA with the rank chosen so that its trainable parameter count matches your MixLoRA, and plain LoRA with the rank matched to the parameters active per token.*
5. *Report the task metrics you used in NLP Assignments #2 and #3, validation loss, per-layer expert load and MaxVio, training time per step and peak memory.*

Success criteria:

- *A table comparing MixLoRA (three balancing variants) with the three LoRA baselines on quality, trainable parameters, time per step and memory.*
- *A short write-up answering: does the mixture beat a parameter-matched LoRA on your task, and what happens to the router without balancing?*

NOTES:

- *The routed adapters cannot be merged into the base weights after training, because the selection depends on the input. Save the router and expert weights yourself: generic adapter saving utilities may silently drop modules they do not know.*
- *MixLoRA: Enhancing Large Language Models Fine-Tuning with LoRA-based Mixture of Experts (Li et al., 2024): https://arxiv.org/abs/2404.15159, code: https://github.com/TUDB-Labs/MixLoRA*
- *LoRAMoE (Dou et al., 2023): https://arxiv.org/abs/2312.09979*
- *HydraLoRA (Tian et al., 2024): https://arxiv.org/abs/2404.19245*
- *Pushing Mixture of Experts to the Limit (Zadouri et al., 2023): https://arxiv.org/abs/2309.05444*

Grading Policy and Defense

Grading Policy is outlined in the table

Criterion	***Points***
Overall correctness and completeness of the submitted code for the lab assignment, overall quality (metrics) of the proposed solution	*13*
Practical question about the submitted code during defense	*2*
Theoretical part during defense (2 questions + 1 exercise)	*5*
Optional Part 3: implementation, parameter-matched comparison and analysis	*+5 (bonus)*
Criterion	***Points***

Parts 1 and 2 carry equal weight within the 13 points for the code and the quality of the solution.

The defense will be conducted during a practical session, where the Lecturer will evaluate your code, report, and presentation skills. It is important to emphasize that the assessment will not be limited to metrics, analysis, and the performance of your models, but will also take into account the overall quality of your code, including readability, structure, and adherence to good practices. During the defense, the Lecturer will evaluate each student's individual contribution, demonstrated skills, and depth of knowledge. In addition, up to 2 theoretical questions may be asked; these will be based on the questions outlined in Appendix 1 and 1 Exercise from the Appendix 2, but will not necessarily be restricted to them.

AI Tools usage policy

Scope

This policy applies to all generative AI tools that produce or modify text, code, images, or other digital content. This includes, but is not limited to:

- *Text generation tools: ChatGPT, Claude, Gemini, Mistral, LLaMA-based models, etc.*

- *Code generation tools: GitHub Copilot, OpenAI Codex, TabNine, etc.*
- *Language assistance tools: Grammarly, DeepL, Google Translate, etc.*
- *Local LLMs or self-hosted AI models.*

Allowed Uses

You may use AI tools for the following purposes, provided they comply with the conditions below:

- *Spell Check & Style Correction. Example: Using Grammarly to correct grammar and improve style in a lab report or paper review.*
- *Translation. Example: Using DeepL or Google Translate to translate sections of a document into another language.*
- *Idea Generation & Pitching. Example: Brainstorming possible approaches for solving a problem or generating research ideas.*
- *Self-Education. Example: Using ChatGPT to explain an unfamiliar algorithm or concept.*
- *Library & Documentation Exploration. Example: Asking AI to summarize PyTorch documentation for a specific class or function.*
- *Code Assistance (Partial). Writing or correcting specific functions or methods (not entire solutions). Examples: Write a function that reads data and converts it to another format. Implement a data collator with padding to the longest sequence. Create an image augmentation function for training.*

Conditions for Use

Full Transparency:

- *Any use of AI tools must be disclosed in your submission.*
- *Include all prompts, responses, and/or screenshots showing your interaction with the AI tool.*
- *If using online tools, include chat links (if available) or full transcripts in an appendix of your report.*

Partial Contribution Only:

- *AI tools should support your work — they must not produce the complete assignment, lab, or report.*
- *You must be able to explain and defend any AI-generated content during oral defense or Q&A*

Examples

Acceptable Uses:

- *"Correct grammar and style in this paragraph of my report."*
- *"Write a function to load images from a directory and resize them."*
- *"Suggest 3 ideas for improving model generalization."*

Unacceptable Uses:

- *"Write the entire peer review for this paper."*
- *"Generate the complete solution code for this assignment task."*

Appendix 1 (Theoretical Questions)

- *Why is the prefill phase usually compute-bound while the decode phase is memory-bandwidth-bound? Explain using arithmetic intensity.*
- *Write the formula for the KV-cache size of a decoder with grouped-query attention. Which configuration fields enter it, and which (e.g. hidden size, feed-forward size, number of query heads) do not?*
- *Why can two models of very different parameter counts have exactly the same KV-cache size per token? What would you change in the config to halve it?*
- *Explain multi-head, multi-query and grouped-query attention as points on one axis. What is traded for the smaller cache?*
- *What does multi-head latent attention cache per token, and why does it need projection absorption to realise the saving at inference time?*
- *What is the wavelength of a rotary pair, and why are the pairs whose wavelength exceeds the training length the ones that fail at extrapolation?*
- *Why does plain extrapolation of a RoPE model produce a loss cliff rather than a gradual slope beyond the training length?*
- *Position interpolation compresses positions by a factor s. Why does it hurt the resolution of short distances, and why does it usually need fine-tuning?*
- *What does NTK-aware base scaling keep unchanged compared to position interpolation, and why does that make it usable without fine-tuning for modest s?*
- *What is the difference between the advertised and the effective context length of a model? How do RULER-style benchmarks measure the effective length?*
- *Why does a Ukrainian document of the same content use a larger share of the context window than its English translation, and how does this affect a "context length" claim?*
- *What is NoPE, and why can a causal decoder without positional encodings still infer token positions?*
- *Why do production models that use NoPE interleave NoPE layers with RoPE or sliding-window layers instead of dropping positional encodings everywhere?*
- *Why does a mixture-of-experts layer replace the feed-forward block and not the attention block?*
- *What is the difference between total and active parameters? Why is a mixture-of-experts model cheap in FLOPs per token but not in memory?*
- *What is routing collapse, and why is it self-reinforcing during training?*
- *Why is the fraction of tokens routed to an expert not differentiable, and how does the auxiliary loss work around it?*
- *Why does the auxiliary load-balancing loss interfere with the language-modeling objective? How does this show up as a trade-off between balance and perplexity?*
- *What does the router z-loss penalize, and why does it improve numerical stability in low precision?*
- *Why does the sign-based bias update drive the expert loads toward balance? What happens if its update rate is too large or too small?*
- *Compare softmax and sigmoid router gates: how do the expert scores interact in each case when one expert's score changes?*
- *What problems do fine-grained experts and shared experts address?*
- *Define MaxVio. Why do you need the MaxVio of a random router on the same number of tokens before interpreting a measured value?*
- *Why might an English-centric mixture-of-experts model route Ukrainian tokens differently from English tokens, and in which layers would you expect the difference to be largest?*

Appendix 2 (Practical Exercises)

Every exercise asks for a derivation, not only for the final formula. Use the notation of the lectures and state every assumption you make.

1. **Multi-head latent attention: caching only the latent.**

   Lecture 1 – Anatomy of a Modern Decoder and the Economics of Attention (multi-head latent attention, projection absorption)

   Multi-head latent attention caches one latent vector per token, obtained with a down-projection, $c_t = W^{DKV} h_t \in \mathbb{R}^{d_c}$, and reconstructs the key and value of head i with up-projections, $k_{t,i} = W_i^{UK} c_t$ and $v_{t,i} = W_i^{UV} c_t$. Ignore positional encodings.

   (a) *Keys.* Prove that every attention score can be computed from the cached latents alone.

   (b) *Values.* Prove that the layer output can be computed without materializing any value vector.

2. **RoPE as a sum of cosines: what is learned and what is fixed.**

   Lecture 1 (rotary positional embeddings); Lecture 2 – Long Context: Positional Encoding, Sparse Attention and Evaluation (RoPE in complex form, the frequency ladder)

   Let $q, k \in \mathbb{R}^d$ be a query at position m and a key at position n, with d even. RoPE applies $R_m = \operatorname{diag}\big(R(m\theta_0), \ldots, R(m\theta_{d/2-1})\big)$, where $R(\phi)$ is the 2×2 rotation by angle ϕ, acting on the pairs (x_{2i}, x_{2i+1}), and $\theta_i = b^{-2i/d}$ for a base b.

   (a) Prove that $\langle R_m q, R_n k \rangle = \sum_{i=0}^{d/2-1} \langle q_i,\, R\big((n-m)\theta_i\big)\, k_i \rangle$, where $q_i = (q_{2i}, q_{2i+1})$ and k_i is defined in the same way. Which property of rotation matrices do you use?

   (b) Identify each pair with a complex number, $\tilde{q}_i = q_{2i} + \mathrm{i}\, q_{2i+1}$. Show that rotating a pair by $R(\phi)$ multiplies $\tilde{q}_i$ by $e^{\mathrm{i}\phi}$ and that $\langle x, y \rangle = \operatorname{Re}(\tilde{x}\, \overline{\tilde{y}})$ for $x, y \in \mathbb{R}^2$. Derive

   $$\langle R_m q, R_n k \rangle = \operatorname{Re} \sum_{i=0}^{d/2-1} \tilde{q}_i\, \overline{\tilde{k}_i}\, e^{\mathrm{i}(m-n)\theta_i}.$$

   (c) Writing $\Delta = m - n$, show that the attention logit (before the $1/\sqrt{d}$ scaling) equals

   $$\sum_{i=0}^{d/2-1} a_i \cos(\Delta\theta_i + \varphi_i), \qquad a_i = |\tilde{q}_i|\,|\tilde{k}_i|, \quad \varphi_i = \arg\big(\tilde{q}_i\, \overline{\tilde{k}_i}\big).$$

   Note that this is the form of the *attention logit* between two tokens, not of a token embedding. Verify the identity numerically for random q, k with $d = 16$.

   (d) For each of a_i, φ_i, θ_i and Δ, state whether it is a learnable parameter, a function of the content of the two tokens computed through learnable projections (which ones?), or fixed by the configuration and the positions. Which quantity does a context-extension method such as position interpolation change?

3. **Position interpolation versus NTK-aware scaling.**

Lecture 2 – Long Context: Positional Encoding, Sparse Attention and Evaluation (position interpolation, NTK-aware base scaling, YaRN)

We want to use a model trained with context L_{train} at $L_{\text{new}} = s\,L_{\text{train}}$, $s > 1$. Every method below replaces the frequencies by $\theta_i' = c_i\,\theta_i$ with a profile $c_i \in (0, 1]$.

(a) *Position interpolation (PI)* maps every position $m \mapsto m/s$. Show that this is equivalent to $c_i = 1/s$ for all i, and that at L_{new} the largest phase $\Delta\theta_i'$ of every pair equals the largest phase it reached during training.

(b) *NTK-aware scaling* keeps positions unchanged and replaces the base, $b' = b\,s^{d/(d-2)}$. Derive $c_i = s^{-2i/(d-2)}$. Show that $c_0 = 1$ and $c_{d/2-1} = 1/s$, and explain why the exponent $d/(d-2)$ is exactly the one that makes the lowest frequency behave as in PI. Compute b' for $b = 10^4$, $d = 128$, $s = 4$.

(c) Where are the two methods similar and where do they differ? Summarize in terms of which frequencies are interpolated and which are extrapolated.

4. **NoPE: positional information without positional encodings.**

Lecture 2 – Long Context: Positional Encoding, Sparse Attention and Evaluation (NoPE, interleaved NoPE and RoPE layers)

A NoPE transformer uses no positional encoding of any kind: token embeddings enter the first layer unchanged, and the only difference between an encoder and a decoder is the attention mask.

(a) Show that a NoPE encoder cannot distinguish "the dog bit the man" from "the man bit the dog" after mean pooling, and that the representation of each token does not depend on its position. Why is this fatal for an encoder?

(b) Show that this is different in the decoder-only case. Then give an explicit construction: choose the query and key weights of one head in the first layer so that its attention is uniform over the prefix, and assume that the beginning-of-sequence token has a value vector with a coordinate equal to 1 where all other tokens have 0. Show that the output of this head at position t contains the value $1/t$, so the absolute position is recoverable.

(c) Why do we need NoPE at all? Explain which problem of rotary embeddings beyond the training length it avoids, and why production models interleave NoPE layers with RoPE or sliding-window layers instead of dropping positional encodings everywhere.

5. **Deriving the auxiliary load-balancing loss.**

Lecture 3 – Mixture of Experts (routing collapse, the auxiliary load-balancing loss)

A batch contains T tokens. The router of a layer with N experts computes logits $\ell(x) = W_r x$, probabilities $p(x) = \mathrm{softmax}(\ell(x))$ and selects $S(x) = \mathrm{TopK}(p(x), k)$. Define

$$f_i = \frac{1}{Tk} \sum_x \mathbf{1}\,[\, i \in S(x)\,], \qquad P_i = \frac{1}{T} \sum_x p_i(x), \qquad \mathcal{L}_{\text{aux}} = \alpha\, N \sum_{i=1}^{N} f_i P_i.$$

(a) *The problem.* Explain routing collapse as a feedback loop: using the fact that the language-modeling gradient reaches only the experts in $S(x)$, argue why an expert that is selected slightly more often early in training tends to be selected even more often later.

(b) *The ideal objective.* Show that $\sum_i f_i = 1$ and that $\sum_i f_i^2 \geq 1/N$, with equality if and only if $f_i = 1/N$ for all i. Explain why $\sum_i f_i^2$ cannot be minimized by gradient descent on W_r.

(c) *The surrogate.* Discuss the motivation behind the exact form of $\mathcal{L}_{\text{aux}}$: why it combines f_i with P_i, why it takes the product $f_i P_i$, and why it is scaled by N. Explain why $\mathcal{L}_{\text{aux}}$ is differentiable with respect to the router parameters although f_i is not.